

Computer Programming 143 – Lecture 1

Introduction to module

Faculty of Engineering



Copyright

Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

The team

Convenor: Dr Chantelle van Staden

Gr	Lecturer	TA
1	Prof Herman Engelbrecht	Ms Danel Slabbert
2	Dr Isaac Khobo	Mr Joshua Spies
3	Prof Herman Kamper	Mr Hendre Jacobs Ms
4	Dr Andrew Gill	Caitlin Ridout

- 1 Lectures and Practicals
- 2 Flexible Assessment
- 3 Development Environment and Compiler
- 4 Information Sources
- 5 Communication Channels
- 6 Aim of Module
- 7 Guidelines and Tips

Lectures (compulsory) and group learning (optional)

- The weekly lecture slides will be made available on STEMLearn
- Facilitated group learning sessions will be announced on STEMLearn
- The practicals rely on you having studied the preceding material

Practicals and tests (compulsory)

- All programming must conform to guidelines – *C Programming Style Guide* (learn.sun.ac.za)
- Code handed in is tested for copying; copying / plagiarism is an offense for which you can be suspended from the university; you are responsible for the security of your code!
- Marks queries limited to within 7 days of publication

Morning Practical 11:00 - 13:00		
Group 2	Chemical Engineering	Electronic Classroom E01.2
Group 2	Civil	M2002 & M2004 - Adjoining rooms
Group 4	Mechanical + Forestry & Wood Sciences	M3002
Group 4	Mechatronics	Jan Mouton 2017
Group 4	Additional Mechatronics students	Overflow into the remaining space in M2002/M2004
Afternoon Practical 14:00 - 16:00		
Group 4	Industrial Engineering(K-Z)	M2002
Group 3	Industrial Engineering(A-J)	M2004
Group 3	Electr & Electron & Data	M3002
Group 1	Afrikaans	Jan Mouton (Electr. classr.) 2010

Flexible Assessment

The module uses *Flexible Assessment*

Mark for weekly programming tests

- If more than 2 unsatisfactory practical tests (test missed, not written correct time slot) – **INCOMPLETE**
- Weekly tests mark P_w is average of all marks for the weekly practical tests excluding the 2 worst tests

Semester mark

- Semester mark $SM = \text{Average}(\text{Prac Tests})$

Tests

- 2 Tests A_1 and A_2 (3^{rd} if needed)
- Final mark $= 0.1 \times SM + 0.4 \times A_1 + 0.5 \times A_2$

Repeaters

- Repeaters who have timetable clashes may apply for exemption from lectures. This will allow you to miss a maximum of 2 lectures per week. To apply for this, please fill in the repeater exemption form on STEMlearn and email the form to cvanstaden@sun.ac.za
- Repeaters can apply for exemption from practicals and practical tests
- Repeaters qualify for exemption if:
 - Semester Mark (SM) > 50%
 - Final Mark (after A_3) > 40%
- To apply, please fill in the response quiz on Stemlearn
- We will check to make sure your responses are correct
- **Email confirmation of your exemption will be sent once you've completed the quiz (provided you qualify)**

IDE and compiler

- IDE: Code::Blocks (Windows/Linux/Mac)
- Compiler: Gnu C-Compiler (GCC)
- Installation and instructions for Windows available on learn.sun.ac.za
- Linux/Mac see www.codeblocks.org

Sources

- Textbook: Deitel, P.J. & H.M., *C How to Program*, 9th edition, Pearson, 2022.
- Extra notes will be shared on STEMLearn

Examples and problems

- Problems from textbook (see learn.sun.ac.za)
- Many example problems on learn.sun.ac.za
- Optional problems in assignments

Communication channels

- **There will be no email correspondence**
- All questions must either be:
 - Posted on the appropriate forum
 - Communicated to the lecturers after lecture sessions or during practicals
- The course webpage and forums is available on stemlearn.sun.ac.za

A student who has successfully completed this module can:

- Understand the composition of a computer system
- Interpret a typical engineering problem and develop software to solve it by:
 - *Designing an efficient algorithm that would solve the problem and presenting it as a flow diagram and/or pseudocode*
 - *Implement the algorithm as a computer program*
- Create code that is:
 - *Easily read and understood by third parties*
 - *Well documented*
 - *Modular*
 - *Easily expandable and reusable*
- Implement appropriate data types and structures in programs

A student who has successfully completed this module can:

- Master the following C skills/concepts
 - *The seven control structures*
 - *Effective use of the C standard library functions*
 - *Effective use of arrays, pointers, characters, strings and structures*
 - *Use of user defined libraries*
 - *C Debugging (syntax and logical errors)*
 - *File processing*

Tips

- Workload: 9 hours per week
 - 3 h lectures
 - 2 h practical
 - 4 h self study
- Stay up to date
- Read the textbook
- Use the material on learn.sun.ac.za
- Ask if you are confused
- Program!

Homework

Homework

- 1 Read the study guide (learn.sun.ac.za)
- 2 Read the C programming style guide (learn.sun.ac.za)